

Faster verified math answers

Callosum speedrun report | 3 October 2026

1. Task, experiments, and runner strategy

The objective is to minimize wall time to **18 distinct verified-correct answers out of 30 AIME 2025 questions** on a single A100 PCIe 80GB GPU. The central observation is that reasoning models often derive an answer well before they finish double-checking or repeating their work. Our runner exposes these intermediate candidates to the grader while generation continues, then stops work as soon as correctness is confirmed.

The grader has one global FIFO worker and charges three seconds for every check, including incorrect answers. Its ideal service floor is therefore **54 seconds** for 18 correct checks. Useful optimizations bring candidates forward, reduce unnecessary checks, and prevent the grader from sitting idle. Aggregate GPU tokens per second alone does not capture this objective.

Baseline. Concurrent pass@4 submits 120 independent requests: four samples for each of 30 questions, with a 16K total context window including the prompt. Only the last integer box in a naturally completed final response is eligible for grading; token-capped outputs are ungraded. Correct verdicts cancel sibling requests, and the eighteenth success cancels all remaining work. A completed BF16 baseline reached 18 in 336.497 seconds; a later interrupted rerun is excluded from ranking. [9]

Core policy. The coverage-first runner starts one streaming rollout per question, initially allowing 8,192 generated tokens. The prompt asks the model to emit prospective boxed answers as soon as they become available. A CPU parser watches reasoning and content for complete integer boxes, standalone answer lines, and recognized prose answer clauses. It rejects partial numbers and invalid numeric forms; network chunk boundaries never delimit an answer.

Each question maintains a candidate queue and a normalized set of already proposed integers, shared across samples and continuation segments. One consumer submits a candidate and waits for its verdict before submitting another for that question. Other questions can each have a pending check. Generation proceeds during local and grader queue waits; a negative verdict leaves the trajectory running. A positive verdict cancels all remaining generation for that question. Distinct first-solved verdicts define progress, with an immediate global stop at 18.

Frozen final strategy. The selected baseline is NVFP4 VibeThinker-3B with Marlin weights, BF16 activations/KV, FlashInfer attention, 95% GPU allocation and 65,536-token total context. All 30 questions start one rollout. Barrier scheduling finishes a coverage round before starting continuations. Each later request allows up to 16,384 additional output tokens, clipped to remaining context; capped unsolved trajectories resume exact IDs, while natural completion or exhausted context can start fresh samples. Four requests per question include continuations. The default uses the cheap 30-stream x 32-token warmup. Frozen core v1 checks source/dependency hashes at startup; prompts and parameter presets have separate recorded hashes. Dynamic60/eight-request allocation remains experimental, not the selected strategy. [1, 10]

What we tried. Beyond streaming extraction and exact-ID continuation, we swept initial fan-out, compared BF16 and NVFP4/Marlin, tested FlashInfer attention, profiled client CPU, deferred trace writes, and tested dynamic60 allocation. Warm vLLM and the sampling batch before timing. Benchmark mode buffers required evidence in RAM and disables optional profiling, then flushes afterward. Initialization, cancellation settlement, and flush are reported separately.

Start here as a teammate. Use `python -m runner_final.run_frozen` with the baseline preset; read the core contract and runner catalogue [1]. Historical canonical and experimental entrypoints remain preserved. The stronger-prompt and 30 x 2 / 4K presets are prepared, untested alternatives; no new sweep was launched during the freeze.

2. What worked and measured results

The complete coverage strategy delivered a measured 3.64x improvement: 92.428s to 18 correct versus 336.497s for naive final-answer pass@4. Both runs used BF16 VibeThinker and an 80% memory budget, with 18 correct checks and no wrong checks. Prompts, concurrency, seeds, generation budgets, continuation, and total context differed, so this compares complete strategies rather than isolating intermediate extraction. [9]

Experimental 60-request ceiling. Prior BF16 sweeps show that wider concurrency raises aggregate throughput while reducing per-trajectory speed. Starting 30 x 1 aims for early useful answers; expanding later to 60 is a guess at a middle ground between 30 and 120. The observations do not identify an optimum, and the hypothesis still needs matched repeated trials. [3]

Times below end at the eighteenth positive verdict, excluding setup and teardown. Paired seeds and initial payloads did not ensure identical realized trajectories.

Recorded configuration	Time to 18	Interpretation
BF16, final-only pass@4 / 16K context	336.497s	Completed naive baseline; 80% memory budget
BF16, coverage + continuation / 8K start	92.428s	Complete strategy comparison; 80% memory budget
BF16, initial 30 x 1 / 8K	71.135s	BF16 sweep best; exact-source replay took 102.99s
BF16, matched 30 x 1 benchmark repeats	134.52s median	Three trials: 117.17-135.65s
NVFP4, matched 30 x 1 benchmark repeats	96.67s median	Three trials: 96.67-115.40s
NVFP4 + FlashInfer, scored validation	86.574s median	87.356 / 59.316 / 86.574s; repeatability gate failed
NVFP4, dynamic60 with growing budgets	127.565s	Single run; several controls changed together
Final v1, five seeds with AIME 2024 warmup	78.601s median	66.481-105.337s; only 1/5 within 71.135s [10]

At 95% memory, the 30 x 1/2/4 sweep reached 18 in 71.135/77.029/114.615s during the first 8K pass. Wider fan-out raised aggregate throughput but reduced observed per-active-request rates to 115.5/94.6/69.7 tok/s. Contexts, cancellations, and active question mix varied; this is not fixed-context scaling. Zero preemptions and at most 20.04% KV occupancy do not support KV exhaustion as the cause of slowdown. [3]

NVFP4 improved observed decoding, but not TTFT. With FLASH_ATTN fixed, three repeats per profile averaged 112.5 versus 136.6 tok/s in initial-stream medians (+21%). Median TTFT averaged 163 versus 205ms, slower with NVFP4. First client grader submission averaged 6.71 versus 4.48s (-33%); an exploratory one-sided Welch test gave p=0.056, above 0.05. Paths and commits differ; these are workload observations. Separately, buffered benchmark mode reduced CPU 52.606 to 37.509 CPU-s without improving first-18 time (106.570 to 106.931s). [4, 5]

Dynamic60 allocation worked; latency did not improve. It admitted 112 fresh requests and 28 continuations, reached 60 streams, and respected eight requests per question. Winners were thirteen initial, four fresh, and one continuation. Extensions only exercised 8K to 16K. Service took 57.002s, with 61.573s idle. Its 127.565s result was slower than the earlier 106.931s NVFP4 16K run. [6]

Seventeen earlier follow-ups before the backend change did not recover 71.135s. FlashInfer development times were 98.327/64.453/64.399s. Scored validation reached 87.356/59.316/86.574s, failing the all-three-below-71.135s gate; settling took 136.048s. The 59.316s run had only 0.483s grader idle time. Validation first submission averaged 4.78s, so the gain chiefly appears in later candidate arrivals. [4]

3. What didn't, next steps, and handoff

What did not improve latency. Increasing initial fan-out from 30 x 1 to 30 x 4 slowed the observed sweep from 71.135s to 114.615s. Dynamic60 ran correctly but took 127.565s versus the earlier 106.931s NVFP4 16K run; it is not a demonstrated latency win. Buffered benchmark mode lowered measured CPU use without improving wall time in its matched pair. Seventeen follow-ups failed to recover the original BF16 timing, showing why a best run is insufficient. These comparisons are qualified by changed trajectories and controls. [3-6]

Approaches we set aside. CPU Gemma salvage was slow and added no grounded candidates in the selected tests; model-only verification falsely approved wrong answers. Prompt-only stopping did not reliably prevent rechecking. Jev confidence pruning and self-consistency did not warrant adoption. Python tools improved hosted-model capability and replay estimates, but VibeThinker tool-use reliability and live latency remain unvalidated. Detailed results stay in Appendix A of the evidence packet, rather than being presented as local VibeThinker measurements. [7, 8]

Use three latency targets to diagnose a run. The actual verification timeline separates first usable candidate arrival, useful grader service, and late idle gaps. For BF16 30 x 1, 5.055s to first pickup + 54.002s service + 12.076s idle approximately equals 71.135s; BF16 30 x 4 had 46.279s of idle gaps. Bring the first candidate forward, keep the grader supplied while limiting wrong checks, and rein in the late-answer tail. Trace CPU grows with fan-out, but CPU-seconds overlap inference and cannot be added to elapsed time. [3, 5]

Next experiments after the freeze. Keep the core fixed and compare prompt adherence first, then 30 x 1 / 8K against 30 x 2 / 4K with exact-ID continuations and the four-request cap. Prior five-seed traces show 10-11 verified questions extracted by 4K versus 14-16 by 8K; their 4K union covers only eleven distinct questions. A second sample may diversify the tail, but an early doubling of solved questions is unsupported. Several late paths spend 6-27s rechecking after the requested result appears. Use predeclared seeds, matched warmup/profiles, all outcomes, and server restarts before claiming repeatability. [10]

Handoff rule. Preserve the baseline and versioned policies. Compare the eighteenth distinct positive verdict, grader service/idle time, wrong checks, and request usage; also retain setup and cleanup costs. Change one control at a time where possible and show all outcomes. Stopping at 18 does not establish full 30-question accuracy or arbitrary-seed repeatability. The evidence packet supplies the figures and source records needed to audit these decisions.

Evidence packet and source index

The [illustrated evidence packet \(PDF\)](#) supplies verification (E1, page 2), concurrency (E2, page 3), CPU (E3, page 4), allocation (E4, page 5), backend outcomes (E5, page 6), and the citation docket (E6, page 7). Additional models stay in Appendix A (page 8); the quantization/first-answer analysis is E7 (page 9).

[1] Policy: Frozen core and presets; historical runner catalogue.

[2] Intermediate replay: Additional-model evidence in Appendix A.

[3] Concurrency: BF16 sweep and throughput.

[4] Replication and backend: All completed comparisons.

[5] CPU and verification: Runner profiling and benchmarking overhead.

[6] Dynamic60: Run and continuation audit.

[7] Alternatives; [8] Python: Additional-model evidence in Appendix A; direct sources in E6.

[9] Live baseline: Completed pass@4 and coverage comparison.

[10] Final validation: All five outcomes, late reasoning, and token distribution.